Modeling and Optimization of Electrodialysis for Nutrient Recovery from Wastewater: *User Manual*

Sai Tarun Ganapavarapu[1], Dylan J. Weber[1], Aditya V. Sankaran[1], Evan Zhou[1], Thi Dao[1], Mohammed Tahmid[1], Hyuck Joo Choi[1], Marta C. Hatzell[1,2], Joseph K. Scott[1]

[1]Department of Chemical & Biomolecular Engineering, Georgia Institute of Technology, 311 Ferst Drive NW, Atlanta, 30332, GA

[2]George W. Woodruff School of Mechanical Engineering, Georgia Institute of Technology, 770 Ferst Drive NW, Atlanta, 30332, GA

1 Optimization (Single Case)

The code base provided in this folder is designed to solve the cost-optimization problem of electrodialysis (ED) for producing a target product of a specified concentration. The implementation is developed in Pyomo (version 6.9.1) and is organized into multiple folders, each containing modular functions. Every function is accompanied by a preamble that clearly describes the function name, objective, inputs, and outputs, thereby improving readability and usability for the user. An illustrative example of such a preamble is presented in Figure 1.

```
#############################################################################
# function:     getDimLessNumbers()                                         #
# Desctiption:  Equations to compute velocity, Reynolds Number, Schmidt     #
#               Number in both concentrate and dilute channels              #
#               Sherwood Numbers at (CEM/Concentrate), (AEM/Concentrate),   #
#               (CEM/Dilute), and (AEM/Dilute) interfaces                   #
#                                                                           #
# Input:        - m : Pyomo concrete model                                  #
#                                                                           #
# Output:       - m with Dimension-less numbers added as components         #
#                                                                           #
#############################################################################
```

Figure 1: The function preamble in Python

1.1 Primary Script - `main.py`

The script **`main.py`** serves as the main driver program for running the electrodialysis (ED) cost-optimization problem. It initializes the optimization model, sets up input parameters, calls all relevant functions from the supporting scripts, executes the optimization routine in Pyomo, and finally retrieves, processes, and stores the results. Additionally, **`main.py`** allows the user to specify outlet concentration targets for both the dilute and the concentrate streams by adjusting the model variables (`m.concOutDiluteND` and `m.concOutConcentrateND`).

1.2 Supporting Functions

Several companion scripts are provided to allow flexibility in defining the optimization scenario and modifying model assumptions. The key scripts users may modify for a specific use case include:

- **`getParams.py`**: Defines process input parameters (e.g., feed concentration, temperature, number of cell pairs, salt properties such as molecular weight and transport numbers, and many more). Users can change these inputs here to simulate different operating conditions.
- **`getVars.py`**: Defines the decision variables for the ED optimization problem. If the user wishes to fix or modify specific decision variables, this script should be updated accordingly.
- **`getCostEqsED.py`**: Contains the cost equations for electrodes and membranes. Users can adjust this script to reflect updated or alternative cost data for these components.
- **`getDimLessNumbers.py`**: Sherwood Number Correlations are implemented within this supporting function. These can be modified if the user prefers to adopt alternative correlations that better reflect their experimental conditions.

2 Optimization (Data Generation)

This version of the code is an extension of the single-case optimization framework described above, with the primary modification introduced in the main script. Users are advised not to modify this version, as it is specifically provided to reproduce the results presented in the paper, particularly the figures in the optimization section that illustrate the variation of decision variables as a function of the product concentration. Unlike the single-case code, users can now provide a list of concentrate outlet concentration targets. The code will automatically execute optimization runs for each target in the list. The script automatically generates an Excel spreadsheet that compiles the results for all specified concentration targets, as well as a single figure comprising four subfigures.

3 Data for Sensitivity Study

The sensitivity study explores the effects of varying feed concentration, dilute concentration, and membrane costs across four cases with increasing product concentration targets. The results for feed concentration and membrane cost variations are presented in the main manuscript, while the sensitivity analysis for dilute concentration is included in the Supporting Information (SI) [add number].

The data for these sensitivity plots is generated using the code base located in the folder "Optimization (Single Case)." All results are compiled in the Excel spreadsheet titled `sensitivityData.xlsx`. After running the optimization problems via **`main.py`**, the cost data for the plots is extracted using FILTER, as demonstrated within the Excel sheet itself.

4 Simulation (Single Case)

The simulation is implemented in MATLAB, leveraging the implicit ODE solver `ode15i`. Similar to the optimization code base, the simulation code is organized into multiple folders containing modular functions. Each function includes a preamble that details its name, purpose, inputs, and outputs. An example of a MATLAB function preamble is shown in Figure 2.

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% function:    getDimLessNumbers(...)                                           %
%                                                                               %
% Description: Compute DimensionLess numbers in the Electrodialysis cell        %
%                                                                               %
% Input:       flowConc - State Variable: Flowrate in concentrate channel       %
%              flowDil  - State Variable: Flowrate in concentrate channel       %
%                                                                               %
% Output:      ReConc    - Reynolds Number in concentrate channel               %
%              ReDil     - Reynolds Number in dilute      channel               %
%              ScConc    - Schmidt Number  in concentrate channel               %
%              ScDil     - Schmidt Number  in dilute      channel               %
%              ShConcCEM - Sherwood Number in concentrate, CEM interface        %
%              ShConcAEM - Sherwood Number in concentrate, AEM interface        %
%              ShDilCEM  - Sherwood Number in dilute     , CEM interface        %
%              ShDilAEM  - Sherwood Number in dilute     , AEM interface        %
%                                                                               %
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

Figure 2: The function preamble in MATLAB

4.1 Primary Script - `main.m`

The **`main.m`** script acts as the primary driver for executing the electrodialysis (ED) simulation. It calls supporting functions and implements three simulation modes as described in the paper. Users can select the desired mode within a `for` loop, which is clearly marked in the code. Upon completing the integration, the script outputs profiles for all state variables and calculates the power consumption in kilowatts.

4.2 Supporting Functions

Input parameters are specified in **`getParams.m`**, where the initial comments summarize the major input data derived from the optimization results. Most other assumptions align with those in the optimization study, but users can adjust them as needed for specific cases. Sherwood number correlations can be updated by editing **`getDimLessNumbers.m`**. Additionally, the simulation incorporates a set of correlations to account for non-ideal osmotic coefficients, equivalent conductivity, and activity coefficients. These effects can be toggled on or off using the switches `toggleOsm`, `toggleAct`, and `toggleEqCond` within **`getParams.m`**.

A further toggle, `toggleIntConc`, allows disabling interfacial concentrations to exclude the impact of concentration potential.

5 Simulation (Data Generation)

This code base extends the single-case simulation by modifying the **`main.m`** and **`getParams.m`** functions. Optimization data for a range of concentrate purity targets (0.2–3.9 wt% N) are stored in a `.mat` file and loaded within **`getParams.m`**. The **`main.m`** script incorporates a `for` loop, enabling simulations to be run for each target in sequence. Upon completion, results are stored in vectors and exported to Excel sheets, with output format depending on the selected mode. These Excel files are then imported into Python, where figures are generated using Matplotlib to ensure consistency with the presentation style of other figures. The script for plotting, **`plottingScript.py`**, is included in this section.

6 Data for Spatial Variation Figures

The Simulation (Single Case) code base is utilized to run four cases with concentrate purities of 1, 2, 3, and 3.5 wt% N. The inputs for these simulations are derived from the 0-D optimal results. The resulting data is compiled in the file `spatialVariationsData.xlsx`, which is subsequently used to generate the figure presented in the 1-D simulations section using Matplotlib.

7 CAPE-OPEN/Aspen Plus Simulation

7.1 CAPE-OPEN Installation & Set-up

An installation of the MATLAB CAPE-OPEN Unit Operation from amsterCHEM is required for the Aspen Plus simulation.

Then, follow the steps below:

1. Open Aspen Plus and navigate to the **Customize** tab.
2. Click **Manage Libraries** and enable the CAPE-OPEN library.

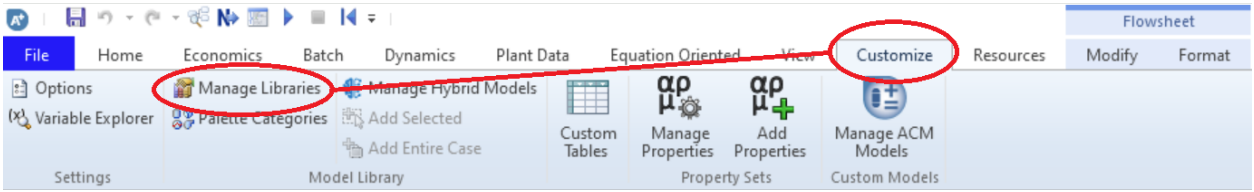

Figure 3: *Steps 1 and 2 of setting up CAPE-OPEN with Aspen Plus.*

3. Close Aspen Plus and then open *ED_SYS_DESIGN_UO.apwz*.

4. If ports are not connected, double-click on the **Matlab Unit Operation** block and click **Load Model**.

   Note: If this is the first time opening the block, there will be a dialogue window to enter the registration key. Enter the key, then click **Load Model**.

5. Select the provided file labeled *PLACEHOLDER NAME.mum*.

6. Connect the disconnected streams to the corresponding ports:

   IN-C to **INLET CONCENTRATE**

   IN-D to **INLET DILUTE**

   OUT-C to **OUTLET CONCENTRATE**

   OUT-D to **OUTLET DILUTE**

   The simulation is now ready to run.

7.2 CAPE-OPEN/Aspen Plus Simulation Interface

Various stream and ED unit conditions can be modified through the Aspen Plus flowsheet and CAPE-OPEN interface.

- **Feed Parameters:** Modify **CAFO-IN** stream.
- **Feed Split:** Modify **FSplit** Aspen block.
- **Electrodialysis Unit Parameters:** Modify **Matlab Unit Operation** through CAPE-OPEN interface.

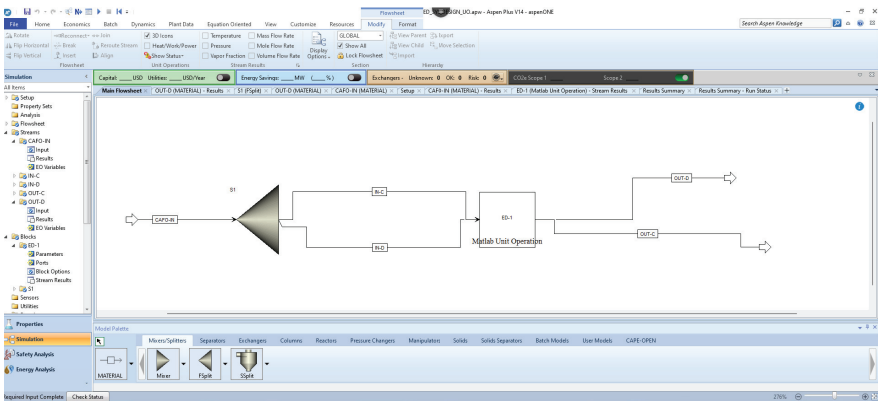

Figure 4: Aspen Plus flowsheet using the custom ED unit operation from CAPE-OPEN.

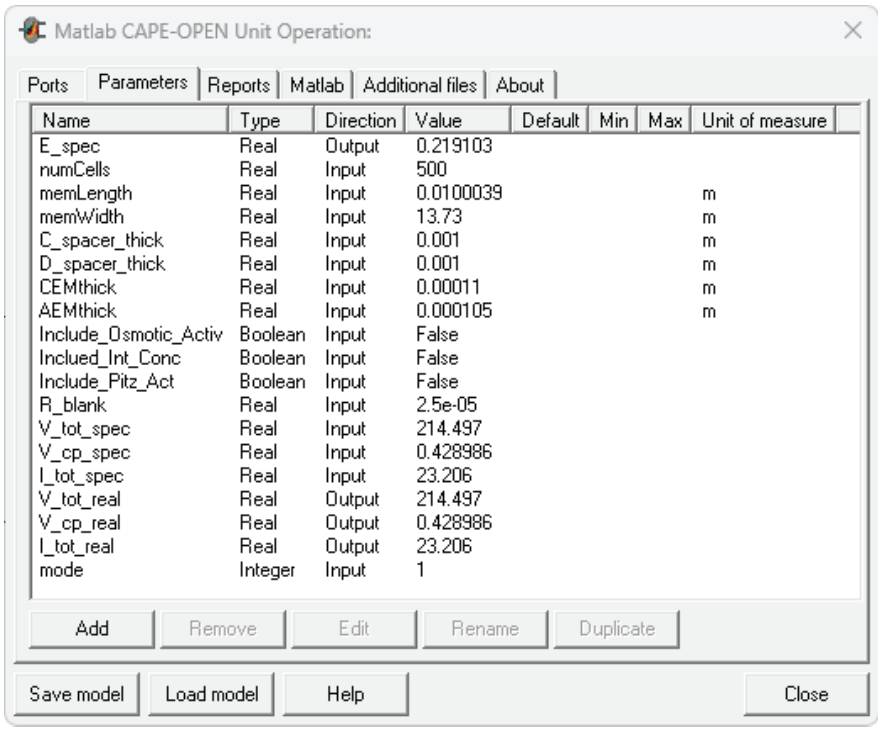

Figure 5: The CAPE-OPEN interface for changing ED block parameters.